



Programación 1

Diseño descendente

Diseño descendente

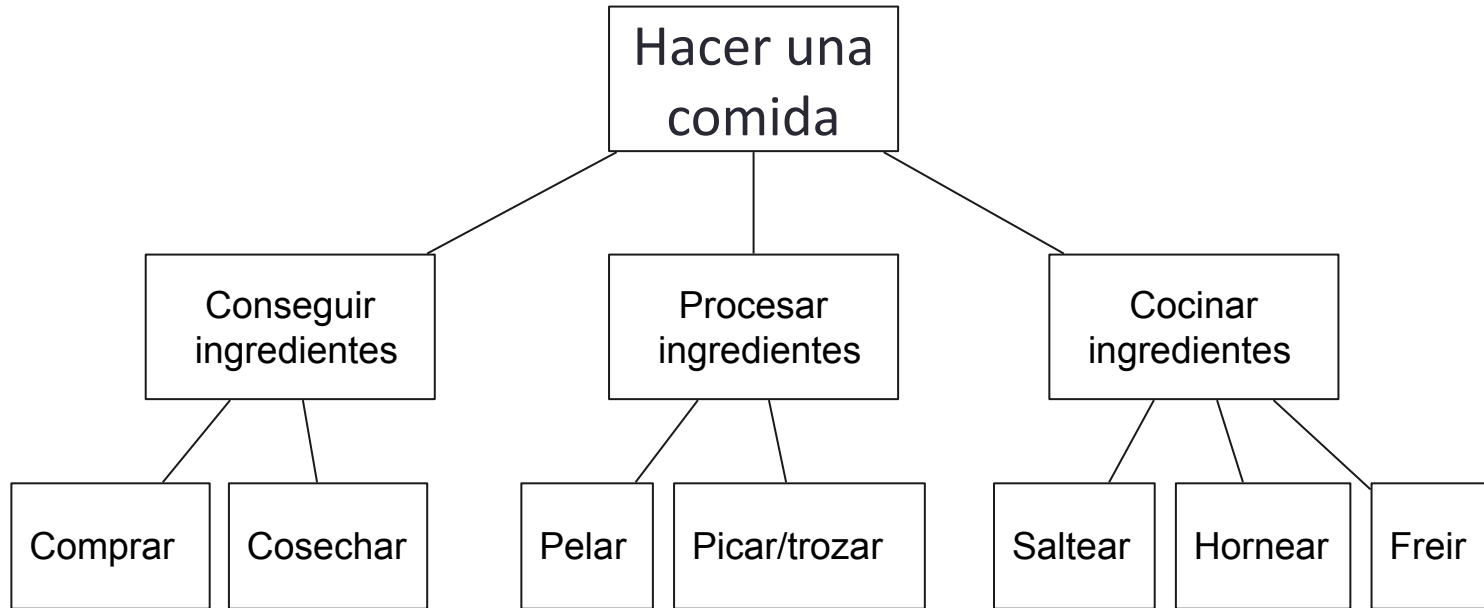
- Uno de los métodos fundamentales para resolver un problema es dividirlo en problemas más chicos o subproblemas.
- Está división se hace repetidamente hasta llegar a pequeños problemas fácilmente solucionables o ya solucionados.
- Es deseable que cada subproblema sea independiente de los restantes.



Ventajas de dividir el problema en subproblemas

- La implementación de un subproblema se puede ejecutar más de una vez **ahorrando tiempo** de programación y **cantidad de código**.
- La división de un problema en subproblemas facilita la **división** de tareas dentro de un grupo de trabajo.
- La resolución de cada subproblema se puede **comprobar individualmente**.
- La división de problemas en subproblemas hace al programa más **legible y modificable**

Diseño descendente



Pseudocódigo 1

La forma más sencilla de describir un problema con subproblemas es anotando los pasos a realizar con oraciones simples en **pseudocódigo**.

Por ejemplo: Escribir un diseño de programa en el que dado un número natural ingresado por el usuario, se chequee si es par y en ese caso se imprima la tabla de multiplicar del 5.

```
public class Clase_3_Ejemplo_1 {  
    public static void main(String args[]) {  
        obtener un número natural por teclado  
        Si es par  
            imprimir tabla de multiplicar del numero 5  
        }  
    }  
}
```

Pseudocódigo 1

```
while (numero <= 0) {  
    // Mensaje de pedido de ingreso  
    System.out.println("Ingrese un número natural:");  
    // Ingreso del valor por teclado  
    numero = Utils.leerInt();  
}
```

```
public class Clase_3_Ejemplo_1 {  
    public static void main(String args[]) {  
        obtener un número natural por teclado  
        Si es par  
            imprimir tabla de multiplicar del numero 5  
    }  
}
```

```
if (numero % 2 == 0){  
    System.out.println("Tabla de multiplicar del 5");  
    for (int i = 1 ; i <= 10; i++) {  
        System.out.println(5*i);  
    }  
}
```

Pseudocódigo 2



Ejemplo: Escribir un diseño de un programa que mientras el usuario ingrese un número entero mayor que 0, imprima la tabla de multiplicar del 10. Cuando salga del ciclo vuelva a imprimir la tabla de multiplicar del 10.

```
public class Clase_3_Ejemplo_2 {  
    public static void main(String[] args){  
        definir número entero  
        obtener un número entero por teclado  
        mientras numero entero sea mayor que 0  
            imprimir tabla de multiplicar de 10  
            obtener un número entero por teclado  
            imprimir tabla de multiplicar de 10  
    }  
}
```

¿Qué es un método?

Un método es una **porción** de **código** al que se lo puede llamar desde el main o desde cualquier otro método. Ej: calcular(); sumar(a,b);mostrar();

Se invoca por su **nombre** seguido de **paréntesis** con o sin parámetros. Estos parámetros son usados por el método para sus cálculos. Ej: sumar(a,b); calcular();

Cuando el método termina, puede o no devolver algún resultado a quien lo llamó.

- Si no **retorna nada** se dice que es un **procedimiento**. Ej: public static void mostrar()
- Si **retorna un valor** se dice que es una **función**. Ej: public static int leerInt(). El tipo del valor retornado debe coincidir con el tipo declarado en la cabecera del método.

Declaración formal de métodos



Signatura del método

```
[modificadores] tipoRetorno nombre ([parámetros formales]) {  
    declaraciones de variables/constantes locales;  
    sentencia_1;  
    ...  
    sentencia_n;  
    [return]  
}
```

Lo que está entre [] no
es obligatorio en todos
los casos

Declaración formal de métodos

modificadores -> ACCESO: **public** - **private** - **protected**

tipoRetorno -> si no retorno nada: **void**

Ejemplo: **public** static **void** mostrar() {

...

}

-> si retorno algo: tipo de lo que retorno

Ejemplo: **public** static **int** leerInt() {

int numero = ...;

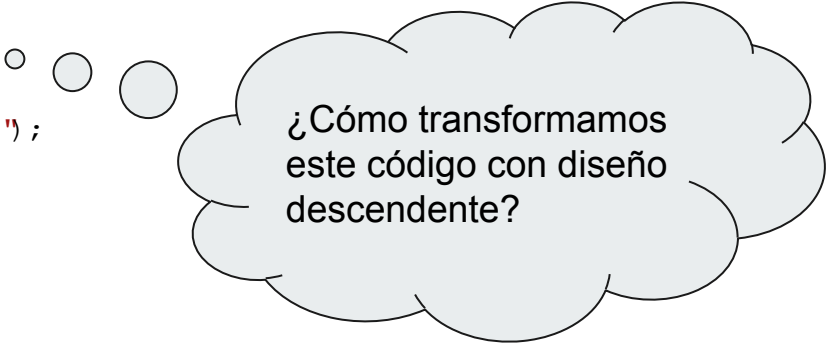
...

return numero;

}

Ejemplo: Si se ingresa un nro natural par, imprime la tabla de multiplicar del 5

```
public class Clase 3 Ejemplo 3 {  
    public static void main(String[] args) {  
        int MAX = 10;  
        int numero = -1;  
        while (numero <= 0) {  
            // Mensaje de pedido de ingreso  
            System.out.println("Ingrese un número natural:");  
            // Ingreso del valor por teclado  
            numero = Utils.leerInt();  
        }  
        if (numero % 2 == 0) {  
            System.out.println("Tabla de multiplicar del 5");  
            for (int i = 1; i <= MAX; i++) {  
                System.out.println(i * 5);  
            }  
        }  
    } // fin del main  
} // fin del class
```



¿Cómo transformamos este código con diseño descendente?

Declaración formal de métodos (procedimiento)

```
public class Clase_3_Ejemplo_3 {  
    public static void main(String args[]) {  
        // damos por obtenido y verificado el número por teclado  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
    public static void imprimirTabla5() { //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

¿Es un procedimiento o función?

Método agregado al Ejercicio

¿Cómo se ejecuta?



1



```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

¿Cómo se ejecuta?

2 →

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

Tabla de multiplicar del 5

¿Cómo se ejecuta?

3 →

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

Tabla de multiplicar del 5

¿Cómo se ejecuta?



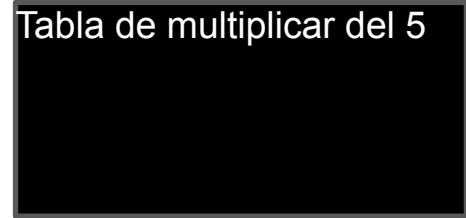
```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
}
```

4 

```
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

Tabla de multiplicar del 5



¿Cómo se ejecuta?

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
}
```

5 →

```
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

Tabla de multiplicar del 5

¿Cómo se ejecuta?

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

6 →

Consola

Tabla de multiplicar del 5
5

¿Cómo se ejecuta?



7..15 →

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

```
Tabla de multiplicar del 5  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```

¿Cómo se ejecuta?

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

16 →

Consola

```
Tabla de multiplicar del 5  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```

¿Cómo se ejecuta?

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

17 →

Consola

```
Tabla de multiplicar del 5  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```

¿Cómo se ejecuta?

18 →

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

Consola

```
Tabla de multiplicar del 5  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```

¿Cómo se ejecuta?

```
public class Clase_3_Ejemplo_3 {  
    public static void main (String [] args){  
        System.out.println("Tabla de multiplicar del 5");  
        imprimirTabla5(); //declaración local  
    }  
  
    public static void imprimirTabla5(){ //declaración formal  
        for (int i = 1 ; i <= 10; i++) {  
            System.out.println(5*i);  
        }  
    }  
}
```

19 →

FIN

Consola

```
Tabla de multiplicar del 5  
5  
10  
15  
20  
25  
30  
35  
40  
45  
50
```



Ejemplo

Solicitar al usuario el ingreso de dos letras minúsculas (a ... z), se debe continuar solicitándoselas hasta que las ingrese correctamente. Si la primera letra es menor o igual a la segunda, informar que están en orden alfabético.

Ámbito de variables



- **Locales:** declarada dentro de un método. Sólo está disponible dentro de su bloque.
- **Globales:** definidas en el bloque que contiene al método que puede usarla.

Ámbito de variables

¿Se pueden tener dos variables con el mismo nombre?

```
public class Clase_3_Ejemplo_4 {  
    public static final int a = 2; // constante global  
    public static int b = 2; // variable global  
    public static void main(String []args) {  
        int a = 3; // local ← accesible sólo dentro de resolver  
        System.out.println ("a = " + a);  
        System.out.println ("b = " + b);  
    }  
}
```

¿Qué imprime por la consola?

Ámbito de variables



```
public class Clase_3_Ejemplo_5 {//BLOQUE del programa Clase_3_Ejemplo_7
    public static int numero = 2;//variable global a todos
    public static void main(String[] args) {//BLOQUE de main
        int a = 3;//variable local a main
        //un bloque se define por {}
        //BLOQUE A
            System.out.println (a + ", " + numero);
            int b = 2; //variable local al BLOQUE A
            System.out.println (a + ", " + b);
            //BLOQUE B
                int c = 3; //variable local al BLOQUE B
                System.out.println (a + ", " + b + ", " + c);
            //FIN BLOQUE B
            //ERROR EN LA PROXIMA SENTENCIA, c SOLO PERTENCE AL BLOQUE B
            System.out.println (a + ", " + b + ", " + c);
        //FIN BLOQUE A
    }//FIN BLOQUE main
}//FIN BLOQUE Clase_3_Ejemplo_7
```

Evitar la definición de bloques internos y variables globales

Definir constantes globales si hay más de un método que necesita usarlas

Definir las variables locales de cada método

Ámbito de variables



```
/*Escribir un programa que llame un método que calcule el promedio de la suma de valores enteros
entre 1 y 1000. Finalmente, el promedio debe mostrarse por pantalla.*/
public class Clase_3_Ejemplo_6 {
    /*Como usamos las mismas constantes en calcular_promedio_1_1000 y en main (en el mensaje de la
    impresión) definimos MAX Y MIN como constantes globales */
    public static final int MAX = 1000, MIN = 1;
    public static void main(String[] args) {
        int promedio;
        promedio = calcular_promedio_1_1000();
        System.out.println("El promedio de la suma entre " + MIN + " y " + MAX + " es "+promedio);
    }
    public static int calcular_promedio_1_1000() {
        int promedio, suma = 0;
        for (int numero = MIN; numero <= MAX; numero++) {
            suma += numero;
        }
        promedio = suma/(MAX-MIN+1);
        return promedio; //LA FUNCION DEBE RETORNAR ALGO DEL MISMO TIPO QUE FIGURA EN LA DECLARACION
    }
}
```



En resumen...

- En el desarrollo de una **solución**, una de las tareas previas a la implementación es el **diseño**, pensar de forma general los pasos que permiten llegar al resultado esperado.
- En el contexto de la materia, un primer **diseño** consiste en obtener el **pseudocódigo** de la solución, que represente un idea de los métodos que se van a tener que implementar.
- Generar un buen diseño permite **abstraerse** del lenguaje, de la implementación.
- Si bien en el examen no se pedirán los **diseños** ni los pasos previos al desarrollo del programa (salvo que se mencione explícitamente), es una práctica recomendable para dar inicio al desarrollo de la solución.
- El diseño con pseudocódigo debería **ayudar** al alumno/a a desarrollar la idea de lo que se desea implementar.

Parámetros



- Un parámetro es una variable cuyo valor es proporcionado por la unidad invocadora.
- Es como un **hueco** donde puedo pasar un **dato** para que lo use otro método.
- Pueden pasarse 1 o más parámetros pero hay que respetar cantidad, posición y tipo.

Ejemplo: Ahora si el número natural ingresado es par, debe mostrarse la tabla de dicho número

```
public class Clase_3_Ejemplo_7 {  
    public static void main(String[] args){  
        int n; // variable local a resolver  
        n = obtenerNatural();  
        if (esPar(n))  
            imprimirTabla(n);  
    }  
}
```

Parámetros



```
public static int obtenerNatural() {  
    // Definición de variables  
    int numero = -1;  
  
    // Proceso de pedir al usuario que ingrese un número natural  
    while (numero <= 0) { // Mientras no ingrese el número natural  
        System.out.println("Ingrese un número natural:");  
        numero = Utils.leerInt();  
    }  
  
    // Retorno el valor ingresado  
    return numero;  
}
```

Parámetros



```
public static void imprimirTabla5(){//declaración formal
    for (int i = 1 ; i <= 10; i++) {
        System.out.println(5*i);
    }
}
```

¿Cuál es la diferencia entre lo que teníamos y lo nuevo?

Nombre del procedimiento

```
public static void imprimirTabla(int numero) {
    for (int i = 1 ; i <= 10; i++) {
        System.out.println(numero*i);
    }
}
```

Parámetro

Parámetros



```
public static boolean esPar(int numero) {  
    if (numero % 2 == 0)  
        return true;  
    else  
        return false;  
}
```

Hay otras alternativas?

```
public static boolean esPar(int numero) {  
    return numero % 2 == 0;  
}
```

Parámetros



Los métodos sin
parámetros



Realiza una tarea única



Un parámetro es una variable cuyo valor es proporcionado por la
unidad invocadora.



Parámetros en la
declaración de un método



Pueden no haber, tener
uno o más



Las variables que se colocan como parámetro cuando se
invoca un método tienen que coincidir en tipo y orden
según la declaración

Parámetros



- En **java** el pasaje de parámetros es por **copia**.
- Se replica la variable y dentro del método se trabaja sobre la réplica.
- Al finalizar el método se pierde la réplica y la unidad invocadora no percibe cambios en la variable utilizada como parámetro de invocación.

Parámetros

Dos parámetros enteros.

Importante: respetar el orden aunque en este caso son los dos enteros



```
public class Clase_3_Ejemplo_8{  
    public static int funcion(int a, int b) {  
        a = 1; //pierde el valor que tenía como parámetro  
        b = a + 5; //pierde el valor que tenía como parámetro  
        return b;  
    }  
    public static void main(String []args){  
        int m = 3;  
        int n = 7;  
        int x = funcion(m,n);  
        System.out.println("Valor de m: " + m);  
        System.out.println("Valor de n: " + n);  
        System.out.println("Resultado de la función: " + x);  
    }  
}
```

¿Cuál es la salida por consola?

Parámetros

```
public class Clase_3_Ejemplo_9{  
    public static int funcion(int a, int b) {  
        a = 1; //pierde el valor que tenía como parámetro  
        b = a + 5; //pierde el valor que tenía como parámetro  
        return b;  
    }  
    public static void main(String []args){  
        int a = 3;      //no importa que se llamen igual que en método  
        int b = 7;  
        int x = funcion(a,b); // x vale 6  
        System.out.println("Valor de a: " + a); // imprime 3  
        System.out.println("Valor de b: " + b); // imprime 7  
        System.out.println("Resultado de la función: " + x); // imprime 6  
    }  
}
```

Hasta ahora



- Pseudocódigo
- Diseño descendente con métodos: procedimientos y funciones
- Ámbito de una variable: local y global
- Parámetros

Errores comunes en métodos

- Una función **siempre necesita retornar algo (para todas las posibles alternativas de ejecución).**

```
public static (tipo no void) nombre_funcion (...){  
...  
return ...;  
}
```

- Una función cuando se invoca sólo puede estar a la derecha de una asignación, dentro de una expresión lógica o como parámetro de otro método.

```
...  
numero = nombre_funcion(...);  
...  
If (MIN<nombre_funcion(...)){  
...  
}  
...  
System.out.println("El resultado es " + nombre_funcion(...));
```

Flujos de retorno



```
...  
public static boolean esPositivo(int x) {  
    if (x<0)  
        return false;  
    if (x>0)  
        return true;  
  
}  
...
```

Para pensar otro ratito
¿Tiene algún problema esta función?

Flujos de retorno



```
public static boolean esPositivo(int x) {  
    if (x<0)  
        return false;  
    if (x>0)  
        return true;  
    // Error: retorno perdido si x es igual a cero.  
}
```

Flujos de retorno



```
public static boolean esPositivo(int x) {  
    if (x<=0)  
        return false;  
    else  
        return true;  
}
```

Flujos de retorno



```
public static boolean esPositivo(int x) {  
    if (x<=0)  
        return false;  
    else  
        return true;  
}
```

¿Qué cambió?

```
public static boolean esPositivo(int x) {  
    if (x<=0)  
        return false;  
  
    return true;  
}
```

Flujos de retorno

```
public static boolean esPositivo(int x){
    if (x<=0)
        return false;
    else
        return true;
}

public static boolean esPositivo(int x) {
    if (x<=0)
        return false;
    return true;
}

public static boolean esPositivo(int x) {
    return (x>0);
}
```

¿Y ahora?

Para resolver



Realizar un programa que dado un carácter ingresado desde teclado (**a** o **b**) permita realizar dos operaciones entre dos números naturales **N** y **M** ingresados desde teclado también. Las operaciones son:

- _ Si se ingresa **a** obtener la potencia de N elevado a la M.
- _ Si se ingresa **b** obtener la potencia de M elevado a la N.

Un Pseudocódigo posible



```
public class Clase_3_Ejemplo_10 {  
  
    public static void main(String[] args){  
        definir opcion, N y M  
        opcion = obtener caracter a o b()  
        N = obtener número natural()  
        M = obtener número natural()  
        si la opcion es 'a'  
            resolver  $N^M$   
        sino  
            resolver  $M^N$   
    }  
}
```

Una solución posible 1/4

```
public class Clase_3_Ejemplo_10{

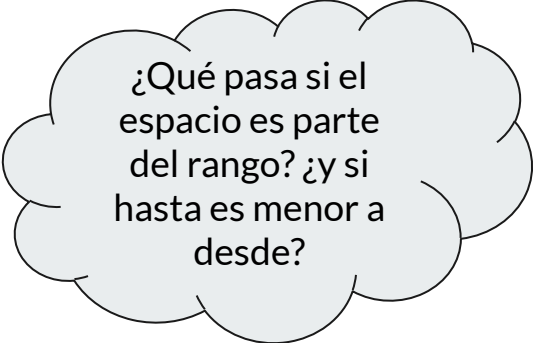
    public static void main(String []args){
        int N = 0, M = 0;
        char opcionDesde = 'a', opcionHasta = 'b';

        char opcion = obtenerOpcionValida(opcionDesde, opcionHasta);

        N = obtenerNatural();
        M = obtenerNatural();
        if (opcion=='a') {
            System.out.println(N + " elevado a la " + M + " es " + calcularPotencia(N, M));
        }
        else {
            System.out.println(M + " elevado a la " + N + " es " + calcularPotencia(M, N));
        }
    }
}
```

Una solución posible 2/4

```
public static char obtenerOpcionValida(char desde, char hasta) {  
    // Definición variables locales  
    char letra = ' ';  
  
    // Mientras no ingrese una opción válida, se la pido  
    while (letra < desde || hasta < letra) {  
        System.out.println("Ingrese una opción entre " + desde + " y " + hasta + ":");  
        letra = Utils.leerChar();  
    }  
  
    // Retornar la opción minúscula ingresada  
    return letra;  
}
```



¿Qué pasa si el espacio es parte del rango? ¿y si hasta es menor a desde?

Una solución posible 3/4



```
public static char obtenerOpcionValida(char desde, char hasta) {  
    // Definicion variables locales  
    char opcion;  
  
    // Controlamos datos pasados como parametro  
    if (hasta < desde) {  
        char aux = desde;  
        desde = hasta;  
        hasta = aux;  
    }  
  
    // Garantiza que no está en el rango desde - hasta  
    opcion = (char)(desde - 1);  
  
    ...  
}
```

Una solución posible 3/4



```
public static int calcularPotencia(int base, int exponente) {  
    int resultado = 1;  
    for (int i = 1; i <= exponente; i++) {  
        resultado *= base;  
    }  
  
    return resultado;  
}
```

Puede salirse de rango si base y exponente son valores grandes.

Parametrización ++

```
public class Clase_3_Ejemplo_10{  
  
    public static void main(String []args){  
        int N = 0, M = 0;  
        char opcionDesde = 'a', opcionHasta = 'b';  
  
        char opcion = obtenerOpcionValida(opcionDesde, opcionHasta);  
  
        N = obtenerNatural();  
        M = obtenerNatural();  
        if (opcion=='a') {  
            System.out.println(N + " elevado a la " + M + " es " + calcularPotencia(N, M));  
        }  
        else {  
            System.out.println(M + " elevado a la " + N + " es " + calcularPotencia(M, N));  
        }  
    }  
}
```

¿Cómo reemplazarían esta parte con un método?

Parametrización ++



```
...  
if (opcion=='a') {  
    informarResultado(N, M);  
}  
else {  
    informarResultado(M, N);  
}  
}  
  
public static void informarResultado(int a, int b) {  
    System.out.println(a + " elevado a la " + b + " es " + calcularPotencia(a, b));  
}
```

Algunos tips



- Usar pseudocódigo para detallar a grandes rasgos las soluciones a los problemas planteados
- Tratar de dividir el problema en subproblemas más pequeños que sean más sencillos de resolver (Modularizar las soluciones)
- Los métodos que retornan algo son funciones y los void son procedimientos
- Usar parámetros para reutilizar las soluciones desarrolladas
- Prestar atención en el llamado a métodos ya que si llevan parámetros deben respetar la cantidad y la posición.
- Si el método, en su definición formal, tiene más de un parámetro y son de diferentes tipos, cuando se llama se debe respetar el tipo que corresponde a cada parámetro. Si no se respeta puede generar errores sintácticos.
- En caso de tener algunos o todos los parámetros del mismo tipo hay que prestar atención como se hace el llamado ya que si me equivoco el orden y respeto los tipos puede ser un error semántico y eso no lo detecta ningún entorno.
- Siempre que llamo a una función debo guardar el retorno en una variable del mismo tipo o hacer uso del resultado para poner en una expresión lógica o como parámetro para algún método o simplemente como salida en la consola.
- Declarar variables lo más cerca de donde se van a usar (locales), evitar el uso de globales y por ahora solo **constantes** globales.